

Modele WMA i VLA w robotach humanoidalnych na przykładzie Unitree G1 EDU

Streszczenie wykonawcze

Modele **WMA (world-model-action)** i **VLA (Vision-Language-Action)** to dwa podejścia do „uczenia sterowania” robotem humanoidalnym, które różnią się tym, **gdzie** powstaje „inteligencja” i **jak** dochodzi do decyzji ruchu. WMA można myśleć jak o **„wewnętrznym symulatorze/predyktorze przyszłości”**: model świata przewiduje, co się stanie po wykonaniu danych akcji, a część „action” wykorzystuje te przewidywania do lepszego wyboru ruchu lub do wytwarzania danych treningowych. W przypadku Unitree jest to np. **UnifoLM-WMA-0**, gdzie świat-model działa jako **Simulation Engine** (generowanie syntetycznych danych) i jako **Policy Enhancement** (wspieranie podejmowania decyzji przez przewidywanie przyszłych interakcji).

VLA to podejście „bardziej bezpośrednie”: model dostaje **obraz (vision)** i **instrukcję (language)** i zwraca **sekwencję akcji (action)** – często jako „pakiet” kilku kroków sterowania (tzw. action chunk). Unitree unifikuje to podejście w **UnifoLM-VLA-0** i opisuje je jako drogę od samego „rozumienia obrazu i tekstu” (VLM) do „ucieleśnionego mózgu” zdolnego do fizycznej interakcji.

Dla **Unitree G1 EDU** praktyczny wybór wygląda następująco:

jeśli celem jest **szybkie uruchomienie wykonywania poleceń/manipulacji** (np. „posprzątaj stół”, „złóż ręcznik”), zwykle zaczyna się od VLA (polityka end-to-end). Jeśli celem jest **planowanie, predykcja skutków, generowanie danych, długa sekwencja zachowań** i/lub wsparcie decyzji poprzez „wyobrażanie” przyszłości, naturalnym kandydatem jest WMA. Unitree udostępnia kody wdrożeniowe WMA w trybie klient-serwer (inference na serwerze, robot jako klient), co jest istotne, gdy robot nie ma wystarczającej mocy obliczeniowej na pokładzie.

Ten dokument rozwija szkic raportu (plik dostarczony w rozmowie) w kierunku praktycznego przewodnika wdrożeniowego.

Metodologia i źródła

Włączone konektory (użyte na początku badań): **Hugging Face** (wyszukiwanie modeli/datasetów Unitree oraz dokumentacji LeRobot). Następnie uzupełniono materiał o publiczne źródła web: oficjalne repozytoria Unitree na GitHub, karty modeli/datasetów na Hugging Face oraz wybrane oryginalne prace naukowe i dokumentację (np. format danych LeRobotDataset v3.0).

Źródła startowe wskazane przez Ciebie (strony projektu UniFoLM WMA i VLA) zostały przeanalizowane poprzez ich publicznie dostępne treści i cytowane poniżej: opis WMA obejmuje dwa tryby (decision-making i simulation) oraz ideę „świat-modelu jako symulatora i wzmacniacza polityki”, a opis VLA akcentuje trenowanie na danych manipulacyjnych oraz demonstracje na platformie Unitree G1.

Modele WMA i VLA w prostych słowach

WMA (world-model–action) to architektura, w której „mózg” robota zawiera ****model świata****: komponent uczący się, jak wygląda i zmienia się środowisko oraz jak środowisko reaguje na akcje robota. Unitree opisuje UnifoLM-WMA-0 jako architekturę z rdzeniem w postaci world-modelu rozumiejącego fizyczne interakcje robot–środowisko, który działa w dwóch rolach:

****Simulation Engine**** (interaktywny symulator generujący dane syntetyczne) oraz ****Policy Enhancement**** (połączenie z „action head”, który korzysta z predykcji przyszłości do poprawy decyzji).

VLA (Vision–Language–Action) to (najczęściej) ****jedna polityka**** ucząca się mapowania: ***(obrazy + instrukcja)* → *(akcje sterujące)***. W UnifoLM-VLA-0 Unitree podkreśla, że model ma wyjść poza ograniczenia klasycznych VLM w interakcji fizycznej i stać się „embodied brain” dzięki kontynuowanemu pre-treningowi na danych manipulacyjnych.

W praktyce oba podejścia często się łączy: VLA może być szybkim „sterownikiem umiejętności”, a WMA – warstwą „przewidywania i planowania”, która sprawdza konsekwencje i stabilizuje zachowanie (szczególnie w dłuższych horyzontach). Tę ideę (łączenie world modelu i VLA) widać też w innych projektach open-source, np. RynnVLA-002, które jawnie łączy model świata i model akcji w jednej ramie.

Porównanie WMA vs VLA

Cecha	WMA (world-model–action)	VLA (vision–language–action)
Intuicja	„Robot **przewiduje przyszłość** i wybiera ruch”	„Robot **patrzy + czyta polecenie** i od razu generuje ruch”
Typowe wejście	Obraz + stan robota + (opcjonalnie instrukcja) + **kandydackie akcje** do symulacji	Obraz (często multi-view) + stan/proprio + instrukcja
Typowe wyjście	Predykcja przyszłych obserwacji/stanów **lub** wsparcie w wyborze akcji (action head)	Sekwencja akcji (często „chunk”, np. horyzont 16 kroków)
Najmocniejsza strona	Planowanie, długi horyzont, generowanie danych, „co jeśli”	Uniwersalne sterowanie zadaniami z języka, multi-task jedna polityka
Ograniczenia	Ciężkie obliczeniowo; ryzyko błędnych „halucynacji dynamiki”; złożony pipeline	Koszt inferencji i real-time; wrażliwość na dystrybucję danych i perturbacje
Przykład Unitree	UnifoLM-WMA-0 (tryb decision-making i simulation)	UnifoLM-VLA-0 (12 kategorii zadań na G1, jedna polityka)
Dane i format	Repo mówi o treningu na danych w formacie **LeRobot v2.1** (ważne dla przygotowania datasetu)	Dane Unitree na HF opisują m.in. pozycje kamer i 30 Hz rejestracji (przydatne do integracji sensoryki)

Jak „uczą się świata” i „planują akcje” w praktyce

WMA (wg repo UnifoLM-WMA-0) ma jasno opisany plan treningowy: najpierw ****fine-tuning modelu generacji wideo**** jako world-model na dużym zbiorze (Open-X), potem „post-training” w ****decision-making mode****, a następnie (opcjonalnie) w ****simulation mode****.

To ważne, bo sugeruje, że world-model jest w dużej mierze modelem generatywnym przewidującym przebieg interakcji (często w formie predykcji kolejnych klatek/obserwacji), a

planowanie przypomina **MPC / best-of-N**: sprawdzamy kilka wariantów akcji „w wyobraźni” i wybieramy najlepszy.

VLA zwykle nie „planuje przez symulację” (choć może), ale generuje akcje bezpośrednio. Dla UnifoLM-VLA-Base można podejrzeć parametry architektury w `config.yaml`: backbone VLM to **Qwen2.5-VL** (pole `model\_type: qwen2\_5\_vl`), a część akcyjna ma parametry typowe dla generowania sekwencji (np. `action\_horizon: 16`, `future\_action\_window\_size: 15`) oraz elementy sugerujące podejście dyfuzyjne/iteracyjne (`repeated\_diffusion\_steps`, `num\_inference\_timesteps`).

W tym samym configu widać też, że trening przewiduje użycie obrazu z nadgarstka i propriocepcji (`use\_wrist\_image: true`, `use\_proprio: true`).

Architektura i komponenty na przykładzie Unitree G1 EDU

Unitree G1 występuje w wariantach, a **EDU** jest kluczowe dla rozwoju: według sklepu Unitree wersja „G1” nie wspiera „secondary development”, a do customizacji i rozwoju wskazana jest edycja EDU (kontakt sprzedażowy).

Podstawowe parametry mechaniczne (np. 1320×450×200 mm, ok. 35 kg) oraz zakres DoF (G1: 23; G1 EDU: 23–43) Unitree publikuje na stronie produktu.

Jeśli używasz dłoni Dex3-1, to jest to opcjonalny efektor z **7 DoF na dłoń** i możliwością wersji z czujnikami dotykowymi.

Integracja sensoryki w kontekście WMA/VLA

Dla praktycznej integracji z percepcją i stanem robota potrzebujesz co najmniej trzech strumieni:

Obraz z kamer. W datasetach Unitree związanych z UnifoLM-VLA-0 podano m.in.: rozdzielczość 640×480, 30 Hz oraz rozmieszczenie kamer: „wrist-mounted (monocular) + head-mounted (binocular)”.

To jest bardzo użyteczne, bo pokazuje realistyczny układ sensoryki używany do uczenia polityk manipulacji.

IMU / orientacja tułowia. W ekosystemie Unitree SDK2 i symulatorze `unitree\_mujoco` widać, że dla G1 istnieje strumień IMUState na temacie `rt/secondary\_imu` (opisane jako „G1 only”).

To typowy sygnał do stabilizacji, estymacji postawy i bezpiecznego ograniczania ruchu (np. gdy robot traci równowagę).

Propriocepcja i stany silników. W dokumentacji ROS2/SDK Unitree pojawia się pojęcie „lowstate” zawierające m.in. `IMUState` i stany motorów (to kluczowy kanał do pętli sprzężenia zwrotnego).

Przykładowe przepływy danych i architektury

Poniższe diagramy są „wdrożeniowo realistyczne” dla G1 EDU: pokazują rozdzielenie na warstwę sensoryki, model, warstwę bezpieczeństwa i sterowanie niskiego poziomu.

```

```mermaid
flowchart TD
 subgraph Robot_G1_EDU[Unitree G1 EDU - warstwa robota]
 CAM[Kamery: nadgarstek + głowa] --> PRE[Preprocessing obrazu]
 IMU[IMU / orientacja] --> FUS[Agregacja stanu]
 PRO[Proprio: stany stawów/dłoni] --> FUS
 PRE --> OBS[Pakiet obserwacji]
 FUS --> OBS
 SAFE[Bezpieczeństwo: limity, E-Stop, stany awaryjne]
 CTRL[Kontroler niskiego poziomu: stawy / dłonie / locomotion]
 SAFE --> CTRL
 end

 subgraph Server_or_Onboard[Inference: serwer lub komputer pokładowy]
 VLA[VLA: vision+language -> action chunk]
 WMA[WMA: world model -> predykcje przyszłości]
 AH[Action head / selekcja akcji]
 end

 OBS -->|instrukcja tekstowa| VLA
 OBS --> WMA
 WMA --> AH
 VLA --> SAFE
 AH --> SAFE
```

```

Diagram odzwierciedla dwa style podejmowania decyzji: bezpośredni (VLA) oraz planujący (WMA + action head), zgodnie z opisem UnifoLM-WMA-0 (decision-making vs simulation) i UnifoLM-VLA-0 (jedna polityka, multi-task).

```

```mermaid
sequenceDiagram
 autonumber
 participant Cam as Kamera (wrist/head)
 participant State as Proprio/IMU
 participant Node as ROS2/DDS Bridge
 participant Model as VLA/WMA Inference
 participant Safe as Safety Gate
 participant Act as Kontroler (SDK2 / DDS)

 Cam->>Node: obraz 640x480 @30Hz (przykład z datasetów)
 State->>Node: stany stawów + IMU
 Node->>Model: (obrazy, stan, instrukcja)
 Model->>Node: action chunk (np. horyzont 16)
 Node->>Safe: planowany ruch
 Safe->>Act: komendy po ograniczeniach
 Act->>Node: lowstate/feedback
```

```

Parametr „action_horizon: 16” oraz użycie proprio i wrist image wynikają bezpośrednio z konfiguracji UnifoLM-VLA-Base.

Minimalne wymagania sprzętowe i programowe

Wymagania zależą od tego, czy inference robisz na robocie, czy na serwerze.

Minimalny zestaw „wdrożeniowy” (rekomendowany dla dużych modeli)

| Element | Minimum praktyczne | Uzasadnienie / źródło |
|---|---|---|
| Robot | **Unitree G1 EDU** (nie zwykły G1) | Wersja G1 nie wspiera secondary development; EDU jest wskazane do rozwoju. |
| Sterowanie i komunikacja | Unitree **SDK2** (C++ lub Python) + DDS | ``unitree_sdk2`` opisuje środowisko (Ubuntu 20.04, gcc 9.4) i jest główną biblioteką; komunikacja oparta o DDS. |
| Python na robocie/kliencie | Python ≥ 3.8 (praktycznie: 3.10) | ``unitree_sdk2_python`` wskazuje Python ≥ 3.8 i zależności jak `cyclonedds/opencv`; projekty Unitree IL/LeRobot i WMA używają Pythona 3.10. |
| ROS2 (opcjonalnie, ale bardzo użytecznie) | Ubuntu 22.04 + ROS2 Humble (rekomendacja z Unitree ROS2) | ``unitree_ros2`` podaje testowane systemy i rekomenduje Humble na Ubuntu 22.04. |
| Serwer inference (WMA/VLA) | x86_64 + GPU (VRAM zależne od modelu) | Model UnifoLM-VLA-Base ma repo ~19 GB (same pliki wag), co zwykle wymaga GPU/VRAM lub agresywnej kwantyzacji. |

Uwaga o mocy obliczeniowej: nie ma w źródłach „twardego” minimum VRAM/RAM dla UnifoLM-VLA-Base/WMA, więc liczby typu „24 GB VRAM” należy traktować jako *praktyczne heurystyki*, a nie wymagania. Najbardziej pewna informacja to rozmiar repo z wagami (19 GB) oraz fakt, że konfiguracja wspomina ``flash_attention_2``, co zwykle oznacza GPU-centriczny tryb pracy.

Wymagania programowe z repo Unitree

WMA (UnifoLM-WMA-0) – repo podaje konkretną instalację środowiska: Python 3.10.18, ``pinocchio=3.2.0``, ``ffmpeg=7.1.1``, klonowanie z submodułami i instalację ``external/dlmp``. Unitree IL + LeRobot – repo ``unitree_IL_lerobot`` opisuje instalację LeRobot i środowiska, w tym `pinocchio` i `ffmpeg` oraz instalację ``unitree_sdk2_python`` do komunikacji DDS. LeRobotDataset v3.0 – jeśli budujesz własne dane na Hub, v3 zapewnia streaming i redukuje presję na system plików; dokumentacja podkreśla konieczność ``finalize()`` przy tworzeniu datasetu.

Praktyczna implementacja: scenariusze i przykładowy kod

Poniższe przykłady są świadomie „proste” i mają pokazać, jak spiąć sensorykę, model i sterowanie. Interfejsy sterowania (np. konkretne API G1) mogą się różnić wersją

robota/firmware – to część, którą trzeba zweryfikować na Twoim egzemplarzu (patrz sekcja wdrożenia).

Scenariusz podążania za obiektem

Wariant A: ****klasyczny**** (bez VLA/WMA) – dobry jako test plumbing (kamera → decyzja → komenda).

Wariant B: ****VLA-asystowany**** – VLA dostaje obraz i prostą instrukcję „podążaj za X” i generuje akcje.

```
```python
Pseudokod/Python: podążanie za obiektem na podstawie obrazu
Założenie: masz w ROS2 node odbierający obraz (np. /camera/image_raw)
i możesz wysłać komendę prędkości (loco) przez SDK2/DDS lub ROS2 bridge.

import numpy as np

class FollowObjectController:
 def __init__(self, loco_iface):
 self.loco = loco_iface # TODO: adapter do Unitree SDK2 (G1 LocoClient) lub ROS2
 cmd_vel
 self.kp_yaw = 0.8
 self.kp_forward = 0.4

 def detect_target_center(self, rgb_image):
 """
 Najprostsza detekcja: segmentacja koloru / marker ArUco / gotowy detektor.
 Tu: placeholder zwracający (cx, cy) w pikselach lub None.
 """
 return None # TODO

 def step(self, rgb_image):
 h, w = rgb_image.shape[:2]
 target = self.detect_target_center(rgb_image)
 if target is None:
 # Bez celu: zatrzymaj się (bezpiecznie)
 self.loco.set_velocity(vx=0.0, wz=0.0)
 return

 cx, cy = target
 err_x = (cx - w/2) / (w/2) # -1..1
 err_size = 0.0 # TODO: np. z bbox area → dystans

 wz = -self.kp_yaw * err_x
 vx = self.kp_forward * (0.2 - err_size) # prosta regulacja podejścia

 # Safety clamp
 vx = float(np.clip(vx, -0.3, 0.3))
```
```

```

wz = float(np.clip(wz, -0.6, 0.6))

self.loco.set_velocity(vx=vx, wz=wz)
...

```

Jeśli zamiast klasycznej reguły chcesz oprzeć „co zrobić” o VLA, możesz zastąpić `detect\_target\_center` i regulator wywołaniem polityki VLA (obraz + prompt). Sensowne jest wtedy generowanie „chunków” akcji i wykonywanie ich w pętli sterowania (np. 10–20 Hz), co jest spójne z ideą `action\_horizon` i okna przyszłych akcji w konfiguracji UnifoLM-VLA-Base.

Scenariusz reagowania na polecenia tekstowe/głosowe

Najprostszy, wdrożeniowy podział:

- 1) ****Wejście polecenia****: tekst (np. UI) lub mowa→tekst (ASR).
- 2) ****Model****: VLA generuje akcje; WMA może weryfikować/przewidywać skutki.
- 3) ****Bezpieczeństwo****: filtr poleceń, limity ruchu, warunek „robot gotowy”.
- 4) ****Sterowanie****: wysłanie do kontrolera (ramiona, dłonie, locomotion).

W ekosystemie Unitree spotyka się interfejs audio po stronie robota (TTS/ASR). Wg opisu interfejsów wysokopoziomowych SDK2 dla G1 istnieje `AudioClient` obejmujący m.in. TTS i ASR; w praktyce dostępność może zależeć od wersji/serwera usług na robocie. Przykład interakcji audio przez ROS2 (TTS) pokazuje zewnętrzny driver/poradnik: wywołanie usługi z `request\_data: 'volume=...;speak= ...'`.

```

```python
Pseudokod: "tekst -> VLA -> action chunk -> wykonanie"
Zakładamy adaptery: get_observation(), vla_policy(), execute_action()

def vla_loop(vla_policy, robot_iface):
 while True:
 obs = robot_iface.get_observation() # obrazy + proprio + IMU
 instruction = robot_iface.get_latest_instruction() # tekst z UI/ASR

 if not instruction:
 continue

 # VLA zwraca sekwencję akcji (np. 16 kroków)
 action_chunk = vla_policy.predict(obs, instruction)

 # Prosty safety gate
 if not robot_iface.is_safe_to_move():
 robot_iface.stop()
 continue

 for a in action_chunk:
 robot_iface.execute_action(a) # np. docelowe joint deltas / ee deltas
...

```

Jeżeli chcesz użyć WMA jako „kontrolera ryzyka”, możesz wstawić krok: „zasymuluj 2–3 warianty chunków akcji” i wybrać ten, który minimalizuje ryzyko (np. kolizji, wyjścia poza workspace). To jest zgodne z opisem, że world-model w decision-making mode „predicts information about future physical interactions to assist the policy in generating actions”.

## ## Kroki wdrożenia na Unitree G1 EDU

Najbezpieczniejsza ścieżka wdrożenia to: **\*\*symulacja → test sensoryki → test sterowania bez modelu → test modelu na małych prędkościach → dopiero potem pełne zadania\*\***.

## ### Lista zależności i konfiguracja środowiska

Poniższa tabela zbiera „najbardziej powtarzalny” zestaw kroków z repo Unitree.

Warstwa	Co instalujesz/uruchamiasz	Źródło
--- --- ---	SDK i komunikacja	`unitree_sdk2` (Ubuntu 20.04, gcc 9.4) oraz/lub `unitree_sdk2_python` (cyclonedds==0.10.2, opencv)
Środowisko WMA	conda env `unifolm-wma` (Python 3.10.18), `pinocchio=3.2.0`, `ffmpeg=7.1.1`, repo z submoduleami	
Środowisko IL/VLA (LeRobot)	conda env `unitree_lerobot` (Python 3.10), pinocchio, ffmpeg, instalacja LeRobot i `unitree_sdk2_python`	
Dane do VLA	datasety Unitree „UnifoLM-VLA-0 Collection” (np. czyszczenie stołu)	
Dane do WMA	repo WMA wymaga datasetów w formacie LeRobot v2.1; jest skrypt konwersji	
Symulacja (opcjonalnie, ale zalecane)	`unitree_mujoco` (most sim2real, obsługa IMUState `rt/secondary_imu` dla G1)	

## ### Wdrożenie WMA w trybie klient–serwer

Repo UnifoLM-WMA-0 opisuje architekturę, w której inference działa na serwerze, a robot działa jako klient zbierający obserwacje i pytający o akcje.

Kluczowe kroki (wprost z repo) to:

- Serwer: uruchomienie skryptu `run\_real\_eval\_server.sh` po skonfigurowaniu checkpointu i datasetów w YAML.
- Robot-klient: przygotowanie środowiska `unitree\_deploy` i usług/kontrolerów na robocie (tu możliwe, że część wymaga dostępu do dokumentacji producenta lub uprawnień).
- Połączenie: tunel SSH na port usługi inference (przykład: `-L 8000:127.0.0.1:8000`).
- Start klienta: uruchomienie `robot\_client.py` z parametrami m.in. `--action\_horizon 16` i `--control\_freq 15`.

Warto zaznaczyć dwie rzeczy praktyczne:

- **\*\*Model WMA na Hugging Face jest gated\*\*** (wymaga zaakceptowania warunków i udostępnienia danych kontaktowych), co może być realną barierą w pipeline CI/CD.
- Modele Unitree AI (w tym VLA/WMA) są publikowane na licencji **\*\*CC BY-NC-SA 4.0\*\***, co zwykle wyklucza zastosowania komercyjne i wymaga kontroli zgodności prawnej.



### Jak sprawdzić informacje nieokreślone (np. firmware / wersje usług)

W tym raporcie **\*\*nie zakładam\*\*** konkretnej wersji firmware G1 EDU, bo zależy od egzemplarza i aktualizacji OTA. Najbardziej praktyczne sposoby weryfikacji:

- Sprawdź, czy w Twoim strumieniu stanu (np. „lowstate” w używanym bridge/driverze) jest pole `version` – przykładowy opis wiadomości lowstate zawiera `uint32[2] version`.
- Odnieś się do Unitree Document Center, do którego odsyła SDK2 (`support.unitree.com`) – część materiałów może wymagać konta/uprawnień producenta.
- Zrób test funkcji usług: znane są przypadki, w których pewne API mogą zwracać „not implemented on server” (przykład z issue dot. akcji ramion). To nie dowodzi, że u Ciebie będzie tak samo, ale jest dobrą motywacją, by dodać testy diagnostyczne przed uruchomieniem polityk.

## Ograniczenia, ryzyka i środki bezpieczeństwa

Najważniejsze ryzyka przy WMA/VLA na humanoidzie wynikają z: (a) real-time i opóźnień, (b) nieprzewidywalnych zachowań poza rozkładem danych, (c) bezpieczeństwa „wejść” (obrazy/polecenia), (d) ograniczeń integracji sprzętowej.

**\*\*Real-time / opóźnienie inference.\*\*** VLA potrafią być kosztowne obliczeniowo; istnieją prace przyspieszające inference, np. VLA-Cache (konsumuje ciągłość czasową i używa ponownie tokeny wizualne w KV-cache), aby zwiększyć częstotliwość sterowania. W ekosystemie LeRobot pojawiają się też metody stricte wdrożeniowe, jak **\*\*Real-Time Chunking (RTC)\*\***: asynchroniczne generowanie kolejnego chunka, gdy robot wykonuje bieżący, aby uniknąć „pauz/ szarpnięć” przy dużej latencji.

**\*\*Bezpieczeństwo na poziomie percepcji.\*\*** Badania pokazują, że VLA mogą być podatne na ataki/adwersarialne obrazy prowadzące do „zamrożenia” (robot przestaje reagować na instrukcje). FreezeVLA formalizuje ten problem i raportuje wysokie ASR w benchmarkach, co jest argumentem za filtrowaniem wejść wizualnych i dodaniem watchdogów wykonania.

**\*\*Ryzyko „halucynacji świata” w WMA.\*\*** Klasyczne prace o world-modelach pokazują, że politykę można trenować nawet „we śnie” generowanym przez model świata, a potem transferować do realnego środowiska – to potężne, ale wzmacnia potrzebę walidacji sim2real i ograniczeń bezpieczeństwa.

**\*\*Ograniczenia licencyjne i dostępu.\*\*** Modele Unitree (VLA/WMA) są CC BY-NC-SA 4.0, a przynajmniej część artefaktów może być gated (WMA-0-Base). Ponadto nie wszystkie funkcje usług mogą być dostępne w danej konfiguracji robota/serwera usług.

**\*\*Minimalne środki bezpieczeństwa przy wdrożeniu\*\*** (praktyczne i wdrażalne bez „magicznych” API):

- Zewnętrzny E-Stop i procedura „safe stop” w oprogramowaniu (np. watchdog na brak feedbacku lowstate).

- Twarde limity prędkości/siły/pozycji na warstwie `Safety Gate` (nie ufaj wyjściu modelu bezpośrednio). W szczególności dla dłoni z siłowym sterowaniem i czujnikami dotyku.
- „Tryb niskiej energii” podczas pierwszych testów: małe prędkości, duże marginesy workspace, praca z miękkimi obiektami.
- Telemetria i logowanie: nagrywanie obrazów i stanów (w stylu datasetów Unitree: 30 Hz, multimodal) ułatwia diagnostykę i późniejsze douczenie.

## ## Propozycje dalszych badań i ulepszeń

Najbardziej obiecujące (i praktyczne) kierunki rozwoju dla G1 EDU to:

**\*\*Hybryda VLA + WMA\*\***. W wielu zastosowaniach humanoida warto rozdzielić „rozumienie zadania” od „sprawdzenia konsekwencji”: VLA wybiera ruch/umiejętność, a WMA ocenia czy plan jest bezpieczny (np. kolizje, rozlanie, upuszczenie). Takie łączenie modeli świata i modeli akcji jest aktywnym trendem (np. projekty łączące „action world model” i VLA w jednym frameworku).

**\*\*Optymalizacja inference\*\***. Poza RTC i cache’owaniem tokenów, w praktyce możesz też rozważyć: kwantyzację, ograniczanie liczby kamer, zmniejszenie rozdzielczości wejścia (np. do 224×224 jak w configu UnifoLM-VLA-Base), lub heterogeniczne pipeline’y (VLM na serwerze, action expert lokalnie).

**\*\*Rozszerzanie modalności\*\***. Jeśli Twój G1 EDU ma dłonie z dotykiem, możesz rozwijać polityki o sensory dotykowe; Dex3-1 ma warianty z macierzami czujników dotykowych.

**\*\*Skalowanie danych i streaming\*\***. Przy własnych datasetach na HF Hub, LeRobotDataset v3.0 daje streaming i lepsze skalowanie plików; to ułatwia trenowanie większych polityk bez trzymania wszystkiego lokalnie.

**\*\*Lepsze „podstawowe klocki” polityk\*\***. Jeśli Twoje zadania są kontaktowe i wielomodalne, warto znać fundamenty: Diffusion Policy (akcje jako proces dyfuzyjny) oraz ACT (Action Chunking with Transformers). Te prace stanowią bazę dla wielu współczesnych VLA (również w stylu „chunków akcji”).

## ### Linki do repozytoriów i modeli

Poniżej przykładowe, użyteczne linki (Hugging Face / GitHub) do materiałów bezpośrednio związanych z UnifoLM, G1 i pipeline’ami wdrożeniowymi:

```text

Hugging Face (Unitree):

<https://huggingface.co/unitreerobotics>

<https://huggingface.co/unitreerobotics/UnifoLM-VLA-Base>

<https://huggingface.co/unitreerobotics/UnifoLM-VLA-Libero>

<https://huggingface.co/unitreerobotics/UnifoLM-WMA-0-Base>

<https://huggingface.co/collections/unitreerobotics/unifolm-vla-0>

Przykładowe datasey G1 (UnifoLM-VLA-0 Collection):

https://huggingface.co/datasets/unitreerobotics/G1_Clean_Table
https://huggingface.co/datasets/unitreerobotics/G1_Fold_Towel
https://huggingface.co/datasets/unitreerobotics/G1_Wipe_Table

GitHub (Unitree):

<https://github.com/unitreerobotics/unifolm-world-model-action>
https://github.com/unitreerobotics/unitree_IL_lerobot
https://github.com/unitreerobotics/unitree_sdk2
https://github.com/unitreerobotics/unitree_sdk2_python
https://github.com/unitreerobotics/unitree_mujoco

LeRobot (docs):

<https://huggingface.co/docs/lerobot/en/lerobot-dataset-v3>
<https://huggingface.github.io/lerobot/>

Prace / implementacje referencyjne (VLA i bezpieczeństwo):

<https://github.com/openvla/openvla>
<https://huggingface.co/papers/2307.15818> (RT-2)
<https://huggingface.co/papers/2509.19870> (FreezeVLA)
...

Wskazówka praktyczna: jeśli planujesz „prawdziwe wdrożenie” na G1 EDU, zacznij od repo `unitree\_IL\_lerobot` (łatwiej przejść ścieżkę: dane → trening → test) oraz od WMA repo (gotowy schemat klient–serwer). Dopiero potem rozbudowuj o hybrydę WMA+VLA i optymalizację real-time.